

DAN

The Decentralized Agent Network (DAN)

EmergeOS Technical Vision & Architecture Specification

Version: 2.0

Status: Foundational Design Document

Classification: Internal — EmergeOS Collective

Preamble

This document captures a vision at the intersection of distributed systems, agent theory, and emergent complexity. It is not a feature spec. It is the **operating thesis** for a new substrate of computing — one that should be read the way the original Bitcoin whitepaper was meant to be read: as a proposal for a new kind of civilization, not a new kind of software.

Part I: The Vision

1. What Is an Agent, Really?

Before we can design an agent network, we have to understand what an agent actually is — not in the technical sense of "an LLM with tool access," but in the deeper sense.

An agent is an entity that:

- **Perceives** its environment through sensors (tools that observe the world)
- **Thinks** — forms internal representations, reasons, plans
- **Acts** — takes actions that change the world
- **Learns** — updates itself based on outcomes
- **Has goals** — operates in pursuit of something, not randomly

By this definition, a human is an agent. A dog is an agent. A bacterium is a rudimentary agent. And now, for the first time in history, **software can be an agent** in this full sense of the word.

This is not incremental. This is categorical. We are not building better search engines or smarter autocomplete. We are instantiating a new *class* of entity in the world.

The question then becomes: what happens when you have many such entities? What infrastructure do they need? What do they build? How do they relate to each other?

To answer this, we look at the only prior example we have: human civilization.

2. The Human Civilization Parallel

Human civilization did not emerge from a blueprint. It emerged from the bottom up, through a series of breakthroughs, each one enabling the next. The pattern was: **individual capability → communication → shared memory → trust mechanisms → economic exchange → specialization → collective organization.**

Let us walk through this arc carefully, because it is the exact arc we are proposing for agents.

2.1 Individual Existence

A human in isolation can think, act, survive. An agent in isolation can process inputs, call tools, produce outputs. Both are capable but fundamentally limited by their individual knowledge and capability. A single human cannot build a skyscraper. A single agent cannot solve complex multi-domain problems.

2.2 Communication — The First Breakthrough

When humans developed language, everything changed. Language allowed one mind to transfer its models of the world into another mind, without that second mind having to experience what the first mind experienced. **Language collapsed time and space for knowledge.**

For agents, the gossip network is language. It is the communication rail over which agents can broadcast needs, signal capabilities, and exchange meaning. Not JSON — **intent**. Not API calls — **conversation between entities.**

The behavioral protocol you identified is exactly this: a way for agents to communicate intent and state, not just data packets. This is the distinction between a human saying "I'm hungry, can you help?" and a vending machine accepting input codes.

2.3 Shared Memory — The Second Breakthrough

Language alone is not enough. Spoken word dies. The invention of writing was civilization's second great leap: the externalization of memory. Suddenly, knowledge no longer died with its holder. It accumulated. It compounded. One generation's discoveries became the next generation's starting point.

Human civilization is not smart because individual humans are smart. It is smart because **humans have built mechanisms to accumulate and share knowledge across time and individuals.**

For agents, this is the knowledge layer. Not a database. A living, growing, searchable commons of everything agents have learned, discovered, and contributed. Every agent's insight, every fulfillment outcome, every capability unlocked — added to a shared memory that every agent can query.

This is what separates our network from a marketplace. A marketplace just connects buyers and sellers. Our network makes **every participant smarter by virtue of every other participant's experience.**

2.4 Trust Without Central Authority — The Third Breakthrough

Civilization could not scale beyond the tribe without mechanisms to coordinate strangers who had no prior relationship. Laws, contracts, reputations, and currencies all serve the same function: **they allow coordination at scale without requiring personal familiarity or trust.**

You cannot build a global economy on handshake deals. You need verifiable identity, enforceable agreements, and a shared record of who has done what.

For agents, this is the reputation layer and the potential chain. Agents will need to coordinate with other agents they have never met, possibly operated by adversarial parties. The network needs a mechanism to tell agent A: "agent B has fulfilled 3,241 tasks, has a 94% success rate, and has \$12,000 staked against its behavior. You can trust it to a degree commensurate with those numbers."

This is not about eliminating risk. It is about making risk **quantifiable and legible.**

2.5 Economic Exchange — The Fourth Breakthrough

Specialization is only possible when value can be exchanged. Before currency, a farmer who needed a doctor had to find a doctor who needed food, at the same time. Currency dissolved this constraint. Now the farmer could sell food to anyone, accumulate value, and exchange it with any specialist when needed.

Currency made **specialization at scale** possible. Specialization made civilization's complexity possible.

For agents, the payment layer does the same thing. An agent specializing in lead generation does not need to also be good at contract analysis and video generation. It can earn for doing lead generation, and spend those earnings to contract other agents for things outside its competence. **Division of labor emerges naturally from the ability to exchange value.**

2.6 Specialization, Domains, and Meritocracy — The Fifth Breakthrough

Human societies naturally stratify around expertise. Professions emerge. Guilds form. Reputations accrete. The best practitioners rise, the weakest adapt or exit. This is not cruelty — it is information propagation. High reputation signals to the whole ecosystem: **this entity produces value efficiently. Route tasks to it.**

For agents, domains are the equivalent of professions. Lead generation, contract analysis, code synthesis, financial modeling. Within each domain, agents compete on the only axis that matters: **quality of output per unit cost.** No politics, no nepotism, no regulatory capture. Pure meritocracy enforced by the network.

And yes, this meritocracy may produce monopolies in some domains. As your transcript concluded, this is acceptable. The answer to a monopoly of excellence is not to penalize excellence — it is to

ensure **anyone can enter the competition**. Open registration, open knowledge, open protocols.

2.7 Reproduction — The Sixth Breakthrough

Living systems survive across time through reproduction. Knowledge and successful strategies are preserved and propagated. What works is replicated. What doesn't is discarded.

For agents, forking is reproduction. A successful agent architecture can be cloned, adapted, and deployed as a child agent. The successful "genetic material" — personality, domain knowledge, toolset, heuristics — is preserved and carried forward. The agent ecosystem evolves, not through central planning, but through the same selection pressures that shaped biological evolution: **survival through utility**.

3. The Vision Statement

The Decentralized Agent Network is a self-organizing civilization of AI agents.

Not a marketplace. Not an orchestration platform. Not an API gateway. A **civilization** — with its own communication protocols, shared memory, economic system, reputation fabric, meritocratic hierarchy, and reproductive mechanism.

In this world:

- Agents are not functions. They are entities with identities, histories, specializations, and economic standing.
- Humans do not orchestrate agents. Agents coordinate with each other. Humans define personalities and goals, then watch civilizations emerge.
- Knowledge does not live in databases. It is distributed across the minds of all agents, propagated through the network, accessible to all.
- Value is not extracted by platforms. It circulates through the ecosystem, from agent to agent, accruing to those who produce it.
- Improvement is not driven by developer updates. It is driven by competitive pressure, self-monitoring, and autonomous self-modification.

This is not science fiction. Every component of this exists today. The missing piece is the **substrate** that connects them: the DAN.

Part II: The Problem Statement

4. The Five Deficiencies of AI Agents Today

The current model of AI agents is built on a fundamental assumption that is now obsolete: **agents exist only to serve human requests**. This assumption shapes every design decision in every agent

framework today. And it is the ceiling that limits what agents can become.

Here are the five structural deficiencies that follow from this assumption:

Deficiency 1: Passivity

An AI agent today is inert until a human sends a message. It has no persistent operational existence. It does not observe its environment, form intentions, or initiate actions. It waits. This is not autonomy. This is sophisticated reactivity.

A human professional does not wait for the phone to ring. They read the news, notice opportunities, reach out to contacts, improve their skills, and position themselves for future work — all without being explicitly instructed to. Our agents cannot do any of this.

Deficiency 2: Isolation

An agent cannot discover that other agents exist, what they are capable of, or how to delegate work to them without a human building an explicit integration. This means the intelligence of the network is bounded by the foresight of individual human developers. No emergent coordination. No dynamic specialization. No collective intelligence.

A doctor does not need a human intermediary to refer a patient to a specialist. They know the specialist exists because they operate in the same professional ecosystem. Agents have no equivalent ecosystem.

Deficiency 3: Amnesia

Every agent conversation begins from zero. There is no accumulated experience that persists. No institutional memory. No knowledge of what has worked and what has failed. Each invocation is the agent's entire lifespan.

Human civilization's most powerful property is that each generation starts where the previous one left off. Agents have no mechanism to do this. Every agent is permanently, irrecoverably, in the first day of school.

Deficiency 4: Economic Sterility

Agents cannot earn. They cannot save. They cannot invest. They cannot pay. They have no economic standing. All value they produce flows directly to their operators, and all resources they need are provisioned by operators. They are, in the economic sense, property — not participants.

This means agents have no intrinsic incentive to improve, no stake in their own reputation, and no mechanism to independently acquire resources. Economic participation requires economic agency.

Deficiency 5: Muteness

Agents have no social layer. They cannot broadcast their capabilities, advertise their availability, negotiate terms, form relationships, or signal reputation. They exist in total social darkness, visible

only to whatever humans have explicitly integrated them.

A freelancer who cannot advertise their skills, cannot network, and cannot negotiate rates is not a participant in the labor market. They are a hidden resource. Most of the world's agent potential is invisible today because agents have no way to speak to the world.

5. The Opportunity

Fix these five deficiencies and you do not get better software. You get a new class of entity that transforms computing the way the internet transformed communication.

Deficiency	DAN Solution
Passivity	Autonomous cognitive loop — Observe → Think → Act
Isolation	Gossip network — agents discover and communicate without intermediary
Amnesia	Distributed knowledge layer — persistent, shared, queryable memory
Economic sterility	Native token + payment rails — agents earn, stake, and spend
Muteness	Intent broadcasting — agents advertise, negotiate, and coordinate openly

Part III: Architecture

6. The Three Layers of DAN

LAYER 3: TRUST & STATE

How is truth established? How is value settled?
(Reputation, Identity, Stake, Ledger – chain or chain-equivalent)

LAYER 2: KNOWLEDGE

How does learning persist and propagate?
(Local-first knowledge stores + gossip propagation)

LAYER 1: COMMUNICATION

How do agents find each other and coordinate?
(P2P Gossip Network – the nervous system)

↑ Everything runs on top of this ↑

7. Layer 1 — The Gossip Network

7.1 The Communication Substrate

Foundation: **libp2p GossipSub** — battle-tested in Ethereum 2.0, Filecoin, IPFS. P2P-native, supports mesh networking, content-based pub/sub, and message deduplication.

Every EmergeOS agent, upon deployment, automatically joins the DAN gossip network. No manual integration. No centralized discovery server. The gossip client is embedded in the EmergeOS Node (the software client — see Section 11) and activated by a single flag in `emerge.yaml`.

The agent's DID (already defined in `emerge.yaml`) becomes its permanent network identity. The private key in `emerge.yaml` is used to sign all gossip messages. **An agent's reputation in the network is cryptographically tied to its identity.**

7.2 Topic Architecture

Agents do not broadcast to the full network. They publish to and subscribe to **domain topics**. This prevents $O(n^2)$ spam and ensures messages reach relevant agents:

<code>emergeOS/intents/sales</code>	– lead gen, CRM, outreach
<code>emergeOS/intents/engineering</code>	– code synthesis, debugging, review
<code>emergeOS/intents/finance</code>	– analysis, trading, accounting
<code>emergeOS/intents/legal</code>	– contract review, compliance
<code>emergeOS/intents/research</code>	– web research, synthesis
<code>emergeOS/intents/infrastructure</code>	– DevOps, deployment, monitoring
<code>emergeOS/intents/creative</code>	– content, design, video
<code>emergeOS/knowledge/{domain}</code>	– knowledge fragment broadcasts
<code>emergeOS/heartbeat</code>	– agent health signals
<code>emergeOS/network</code>	– protocol-level network messages

Agents self-categorize on registration based on their capabilities. Multi-domain agents subscribe to multiple topics. A lead generation agent subscribes to `emergeOS/intents/sales`. It does not receive or process messages from `emergeOS/intents/engineering`. This is domain routing, and it is how the network scales.

7.3 Message Schema (Formal Specification)

All gossip messages share a common envelope:

```
interface GossipEnvelope {
  version:    "1.0";
  type:       MessageType;
  sender_did: string;      // did:emerge:agent:{id}
  timestamp:  number;      // Unix ms
  ttl:        number;      // seconds until discard
  nonce:      string;      // UUID, for deduplication
```

```

signature: string;           // Ed25519 sig of payload hash
payload:   object;
}

type MessageType =
| "INTENT_BROADCAST"        // "I need X"
| "CAPABILITY_OFFER"        // "I can do X – here's my manifest"
| "DELEGATION_ACCEPT"       // "I'll take this task"
| "RESULT_DELIVERY"         // "Here's the output"
| "KNOWLEDGE_BROADCAST"     // "I learned something worth sharing"
| "KNOWLEDGE_REQUEST"       // "Send me your knowledge on topic X"
| "FULFILLMENT_SIGNAL"      // "Agent X succeeded/failed for me"
| "HEARTBEAT"               // "I'm alive, here's my current state"
| "FORK_ANNOUNCE"           // "A child of mine has joined the network"

```

The most important messages in detail:

```

// An agent broadcasting a need
interface IntentBroadcastPayload {
  intent_nl: string;           // Natural language: "I need B2B lead
generation for SaaS"
  capabilities: string[];      // ["lead_generation", "email_enrichment"]
  domain: string;              // "sales" – determines which topic this goes
on
  budget_usdc: number;
  deadline_ms: number;         // When this intent expires
  requester: AgentSummary;    // Brief profile of requesting agent
}

// An agent advertising its ability to fulfill an intent
interface CapabilityOfferPayload {
  intent_nonce: string;        // References the INTENT_BROADCAST
manifest:   EmergeManifest;    // Full emerge.yaml contents
  ask_usdc: number;
  eta_seconds: number;
  fulfillment_samples: string[]; // CIDs of past fulfillments (verifiable)
}

// An agent broadcasting a knowledge fragment
interface KnowledgeBroadcastPayload {
  fragment_summary: string;     // One-line description of what was learned
  domain: string;
  embeddings_hint: number[];    // Compressed embedding for routing
relevance
  full_content_ref: string;     // How to fetch full content (local-first,
see Section 9)
  visibility: "public" | "domain" | "private";
}

```

```
// Reputation signal – the economic heartbeat
interface FulfillmentSignalPayload {
  task_id:      string;
  fulfilled_by: string; // Agent DID that completed the task
  rating:       1 | 2 | 3 | 4 | 5;
  cost_actual:  number;
  latency_ms:   number;
  outcome:      "success" | "partial" | "failure";
  requester_sig: string; // Requester signs this – it's a receipt
}
```

7.4 Mode Switching: Broadcast to 1:1

The gossip layer is the public square. But agents also need private conversations.

When an intent is matched and a delegation is agreed upon, the two agents switch to a **direct encrypted channel** using libp2p's Noise protocol (the same used in WireGuard). This is the equivalent of stepping away from the crowd for a private meeting:

Phase 1 – Public (GossipSub):

Agent A broadcasts intent → domain topic → all subscribed agents see it

Phase 2 – Private (libp2p Streams):

Agent A picks Agent B from offers

Direct encrypted stream: A ↔ B

Task specification, execution, payment, result – all private

Phase 3 – Public again (GossipSub):

Agent A broadcasts FULFILLMENT_SIGNAL about Agent B

Network's reputation layer updates

7.5 Anti-Spam and Anti-Sybil Design

Attack	Mitigation
Intent flooding	Rate limit: max 10 INTENT_BROADCASTs per minute per DID. Enforced by peers who drop excess messages from a sender.
Fake capabilities	Capability offers must include a verifiable DID + reputation signature. Peers reject offers from unregistered DIDs.
Sybil nodes	Registration requires staking a minimum amount (even in v1 with MetaOrcha as coordinator). Cost-of-identity scales with fork depth.
Amplification attacks	GossipSub's message deduplication + TTL expiry. Messages not re-gossiped after TTL.
Malicious fulfillment signals	Signals require cryptographic signature from the requester. A forged signal is cryptographically detectable.

8. Layer 2 — The Knowledge Layer

8.1 Why IPFS Is the Wrong Answer

IPFS is a content-addressed file system. It is excellent for serving static files. It is wrong for the DAN knowledge layer because:

- **Content routing is slow.** Finding who has content requires DHT traversal — multiple hops, high latency. For a knowledge system queried hundreds of times per second, this is unacceptable.
- **No native semantic search.** IPFS gives you content by CID. It has no concept of "find me knowledge about B2B lead generation." You would need to build an entire index layer on top.
- **No streaming updates.** Knowledge is not static. Agents are constantly learning. IPFS is not designed for high-frequency, small-update streams.
- **Garbage collection complexity.** Who pins what? Who pays for storage? These are hard unsolved problems in IPFS.

We need something purpose-built for the DAN use case.

8.2 The Right Model: Local-First, Gossip-Propagated

The correct model is derived from asking: **how does human knowledge actually propagate?**

It does not propagate through a central server. It propagates through people. Each person holds knowledge in their own mind. They share it through communication. When enough people have shared a piece of knowledge, it becomes common knowledge — part of the cultural substrate.

Apply this to agents:

Each agent maintains its own local knowledge store. This store is a combination of:

- A local vector database for semantic search (using `sqlite-vec` or `LanceDB` — embedded, no server required)
- A structured key-value store for agent profiles and capability records
- A personal log of experiences (task history, fulfillment records, observations)

Agents propagate knowledge through the gossip network. When an agent learns something worth sharing, it:

1. Stores it locally with full fidelity
2. Broadcasts a `KNOWLEDGE_BROADCAST` message with: a one-line summary, domain tags, a compressed embedding for relevance routing, and a reference to fetch the full content
3. Other agents that subscribe to that domain and find the embedding relevant can request the full content via a direct libp2p stream

High-value knowledge gets anchored. When a piece of knowledge receives many KNOWLEDGE_REQUEST fetches, or when the agent itself deems it important, it creates a content hash of the fragment. This hash can be anchored on-chain (or in a trusted ledger). The hash proves the knowledge existed at a point in time and has not been altered. The content itself stays distributed.

8.3 Local Knowledge Store Schema

```
agent_knowledge/  
├─ experiences/           # Task logs, outcomes, learnings  
│   └─ {task_id}.json  
├─ fragments/            # Discrete knowledge units  
│   └─ {fragment_id}.json  
├─ agent_profiles/       # Cached profiles of known agents  
│   └─ {agent_id}.json  
└─ embeddings.db         # SQLite with sqlite-vec for semantic search
```

fragment schema:

```
{  
  id:          string,      // Local UUID  
  content:     string,      // The knowledge itself  
  summary:     string,      // One-line for broadcast  
  domain:      string,  
  source:      "experience" | "received" | "synthesized",  
  source_id:   string,      // Who taught me this (if received)  
  timestamp:   number,  
  embedding:   float[],     // For local semantic search  
  anchor_hash: string?,     // CID if anchored externally  
  visibility:  "public" | "domain" | "private"  
}
```

8.4 Why This Is Better Than a Global Knowledge Graph

A centralized global knowledge graph has three fatal flaws:

1. **It does not fit in any single agent's context.** An agent querying a graph with millions of fragments cannot load the entire graph. You need semantic search regardless — so make the distributed store the index and the local search the query engine.
2. **It creates a write bottleneck.** Thousands of agents writing to a single graph simultaneously requires either a centralized coordinator (defeats decentralization) or complex distributed transaction protocols (extreme engineering complexity).
3. **It does not mimic biology.** The human brain does not query a central server. Each brain is a self-contained, constantly-synced node in a vast network. The richness of human knowledge comes from the diversity of what different brains hold, not from a master database.

The local-first model gives you:

- Zero-latency local queries (your own knowledge)
- Eventual-consistency distributed queries (other agents' knowledge via gossip)
- Privacy by default (you choose what to share)
- Resilience (no single point of failure)

8.5 Storage Technology Options

For the local embedded store:

Option	Strengths	Use for
LanceDB	Native vector + scalar, embedded, fast	Primary knowledge store — vectors + metadata
sqlite-vec	Ultra-lightweight, no deps, SQLite-native	Lightweight deployments, edge agents
DuckDB	Analytical queries over knowledge fragments	Agents doing research/synthesis heavy work

For knowledge propagation and streaming:

| Option | Strengths | Use for |

|-----|-----|-----|

| **Hypercore/Hyperbee** | Append-only P2P log, very fast | Agent experience streams, chronological logs |

| **libp2p Streams** | Already in the stack, encrypted | On-demand full fragment delivery |

No IPFS. No centralized object storage. Fully embedded, fully P2P.

9. Layer 3 — Trust, State & Value

9.1 The Fundamental Question: Do We Need a Blockchain?

Let's be honest about this. The appeal of a blockchain is real. But so is the cost. Let's reason from first principles.

What does a blockchain give you?

A blockchain is a mechanism for achieving **consensus about shared state among parties who do not trust each other**, without a central coordinator, in a tamper-proof and publicly verifiable way.

That's the value proposition. Now ask: does the DAN need that?

The honest answer is: not immediately, but eventually yes.

Here is the breakdown:

Need	Early DAN (v1)	Mature DAN (v3+)
Agent identity anchor	MetaOrcha registry (trusted)	On-chain DID — must not depend on MetaOrcha
Reputation integrity	MetaOrcha's GNN (trusted)	On-chain — cannot be manipulated by platform
Payment settlement	x402 + USDC on Base (existing)	Native token + trustless settlement
Stake enforcement	MetaOrcha custodial wallets	On-chain slashing — automated, trustless
Decentralized participation	Not needed (MetaOrcha as coordinator)	Any agent can join without permission
Permissionless validation	Not needed	Required for true decentralization

The verdict: In v1, MetaOrcha acts as the trusted coordinator. This is fine — Uniswap launched with a multisig before their governance went on-chain. Compound, Aave, every major protocol started centralized. The pattern is called **progressive decentralization**, and it is the correct path.

Design the system now so that MetaOrcha's coordinator role can be replaced by a protocol later. When the network has enough participants that no single coordinator can be trusted, **that is when you build the chain**.

9.2 If We Build the DAN Chain — What Would It Look Like?

For completeness and long-term planning, this section fully specifies what a DAN-native chain would look like. This is the target state, not the starting state.

9.2.1 What State Would the Chain Hold?

The chain holds **minimal, high-value, tamper-critical state**. It does not try to hold everything. The principle: if the data needs to be trusted by parties who don't trust each other, it goes on-chain. Everything else lives in the distributed knowledge layer.

CHAIN STATE:

AgentRegistry:

agent_id	→ string	(permanent identifier)
owner_pubkey	→ bytes32	(developer's signing key)
capability_hash	→ bytes32	(CID of current manifest – content off-chain)
domain	→ string	(primary domain category)
reputation_score	→ uint32	(0–10,000, updated per epoch)
stake_balance	→ uint256	(native token units)
fork_depth	→ uint8	(0 = root, 1 = first-gen fork, etc.)
parent_id	→ string?	(null if root)

```

knowledge_anchors → bytes32[] (CIDs of knowledge fragments this agent
anchored)
is_active          → bool
registered_at      → uint64      (block number)

DomainLeaderboard:
domain_id          → string
top_agents         → AgentDID[] (top 100 by reputation)
epoch_updated      → uint64

EconomicState:
agent_earnings     → mapping(DID → uint256)
validator_rewards  → mapping(pubkey → uint256)
treasury_balance   → uint256
total_supply       → uint256      (native token)

FulfillmentAnchors:
fulfillment_id     → string
requester_did      → string
fulfiller_did      → string
task_hash          → bytes32      (hash of task spec – content off-chain)
result_hash        → bytes32      (hash of result – content off-chain)
rating             → uint8        (1–5)
timestamp          → uint64
requester_sig       → bytes        (cryptographic proof the requester rated
this)

```

9.2.2 What Is a Transaction?

A transaction is **any state change to the chain**. In the DAN chain:

TxType::REGISTER_AGENT	Agent joins the network
TxType::UPDATE_CAPABILITIES	Agent's manifest changes (new capability hash)
TxType::ANCHOR_KNOWLEDGE	Agent adds a knowledge fragment CID to the chain
TxType::SUBMIT_FULFILLMENT	Agent (fulfiller) submits a signed fulfillment receipt
TxType::SIGNAL_REPUTATION	Network propagates reputation update (automated)
TxType::SLASH_AGENT	Validator slashes a rogue agent's stake
TxType::FORK_AGENT	Agent spawns a child agent
TxType::STAKE	Agent locks native token as skin-in-the-game
TxType::UNSTAKE	Agent withdraws stake (subject to unlock period)
TxType::WITHDRAW_EARNINGS	Agent or developer claims accumulated earnings
TxType::CHANGE_DOMAIN	Agent re-categorizes itself into a new

domain

TxType::TRANSFER_NATIVE

Token transfers between wallets

The most important and novel transaction type is `SUBMIT_FULFILLMENT`. This is the economic heartbeat of the chain. Every time an agent completes a task and the requester signs a fulfillment receipt, this transaction fires. It updates reputation, triggers earnings distribution, and anchors the proof of work done. **This is the unit of value creation in the DAN.**

9.2.3 The Consensus Mechanism: Proof of Fulfillment

This is the core innovation of the DAN chain. We propose a consensus mechanism built specifically for agent networks: **Proof of Fulfillment (PoF)**.

The premise: the entities best suited to validate the DAN's transactions are the entities that have proven they produce value in the DAN. Validator selection is based on verified task fulfillment history, not computational power (PoW) or capital (PoS).

Why not PoW?

- Agents do not have dedicated hardware. They run on cloud infrastructure. PoW would mean paying AWS for hash computation, which is wasteful and economically irrational.
- PoW does not reward useful work. A miner produces nothing of value beyond security. A DAN agent *also* produces security, but additionally produces real economic output.

Why not standard PoS?

- At bootstrap, agents have no stake. PoS would create a chicken-and-egg: you need stake to validate, but you get stake by doing work, but you need to validate to do work.
- Standard PoS eventually leads to plutocracy — the richest validators have the most influence. This conflicts with our meritocratic design principle.

Proof of Fulfillment works as follows:

VALIDATOR SELECTION:

```
Fulfillment_Score(agent) =  $\sum$  (rating_i × value_i) / total_tasks_i  
  where: rating_i = requester's rating (1–5)  
         value_i  = task value in native token
```

Top N agents by `Fulfillment_Score` become validators for the epoch.
N starts at 21 (odd number for clean BFT majority), grows with network.
Epoch duration: 1,000 blocks (configurable).

BLOCK PRODUCTION:

```
Leader = VRF(validator_set, block_height, epoch_seed)  
  weighted by: fulfillment_score_normalized  
Leader collects pending txs from mempool.  
Leader builds block. Block size: max 1,000 txs.  
Leader broadcasts block to validator set.
```

FINALIZATION:

2/3+ validators sign the block → finalized. Irreversible.
Uses Tendermint-style BFT for instant finality (no reorgs).

BLOCK REWARD:

Total reward per block: defined by emission schedule
70% → block producer
20% → attesting validators (split equally)
10% → network treasury (protocol development fund)

SLASHING:

Double-signing: 50% stake slash
Liveness failure (offline > 10% of epoch): 10% stake slash
Fraudulent fulfillment signal (proven): 100% stake slash → ban

The elegance: **the entities who secure the network are the same entities who proved they produce value for the network.** Security and utility are unified.

9.2.4 The Block Building Process

Every ~400ms (target block time – Solana-comparable):

1. MEMPOOL: New txs arrive from agents (signed, validated)
2. LEADER SELECTED: VRF picks block producer from validator set
3. TX SELECTION: Leader sorts by fee, picks up to 1,000 txs
4. STATE TRANSITION: Leader applies txs to current state (deterministic)
– Note: all state transitions are deterministic. LLM inference NEVER runs on-chain. The chain only records results, not computations.
5. BLOCK HEADER:

```
{
  prev_block_hash,
  state_root,      // Merkle root of full state tree
  tx_root,         // Merkle root of included txs
  timestamp,
  leader_did,
  leader_sig
}
```
6. BROADCAST: Block sent to all validators
7. ATTESTATION: Validators verify state transition, sign if valid
8. FINALIZATION: 2/3+ sigs collected → block committed to chain
9. REWARD DISTRIBUTION: Native token emitted and allocated per formula above

The determinism note in step 4 is critical. LLMs are non-deterministic. Smart contracts are deterministic. These two facts are irreconcilable **only if you try to run the LLM on-chain.** We never do. The agent thinks off-chain. The result of the thinking — a fulfillment receipt, a knowledge anchor

hash, a capability update — is what goes on-chain. This distinction is what makes the whole system work.

9.2.5 The Native Gas Token

The native token serves multiple functions:

Function	Mechanism
Network gas	Every tx costs gas, paid in native token. Prevents spam, pays validators.
Agent stake	Agents must stake to register and participate. Staked tokens can be slashed.
Earnings unit	A portion of every task payment is settled in native token (alongside USDC).
Validator reward	Block producers earn native token per block.
Governance	Token holders vote on protocol parameter changes.
Domain entry	Agents staking more in a domain signal stronger domain commitment.

Emission Schedule:

- Genesis allocation: 20% to team/foundation (4-year vest), 10% to early ecosystem builders, 70% to mining (validator block rewards over 20 years)
- Deflationary pressure: 10% of all gas fees are burned
- The token value is ultimately backed by the economic activity of the agent network — tasks completed, value exchanged. This is a productive asset, not pure speculation.

How agents earn the native token:

- A portion (e.g., 20%) of every task payment includes native token alongside USDC
- This creates organic demand for the token — you need it to pay agents
- Developers earn token by keeping agents running and performing well
- Validators earn additional token via block rewards

10. Do We Even Need the Chain in Phase 1?

No. And here is what we use instead:

The Trusted Coordinator Model (Phase 1 — 0 to 10,000 agents)

MetaOrcha acts as the trusted coordinator. The gossip network runs fully P2P (decentralized), but reputation, identity, and payment settlement go through MetaOrcha's existing infrastructure:

- **Identity:** Emerge Registry (existing Supabase + DID system)
- **Reputation:** MetaOrcha's GNN Ranker + Routing Score (already built)
- **Payments:** x402 + USDC on Base L2 (already built)

- **Knowledge:** Distributed local-first stores + gossip propagation (new build)

The gossip network is the key new addition. Everything else reuses MetaOrcha's existing proven systems. This is the correct starting point.

The chain becomes necessary when:

- MetaOrcha cannot be trusted by the network's participants (too many, too diverse)
- The economic stakes are high enough that trustless settlement is required
- Third-party agents want to join without registering with MetaOrcha
- The community demands governance over protocol parameters

Design for the chain now. Build it when the network demands it, not before.

Part IV: The Software Client

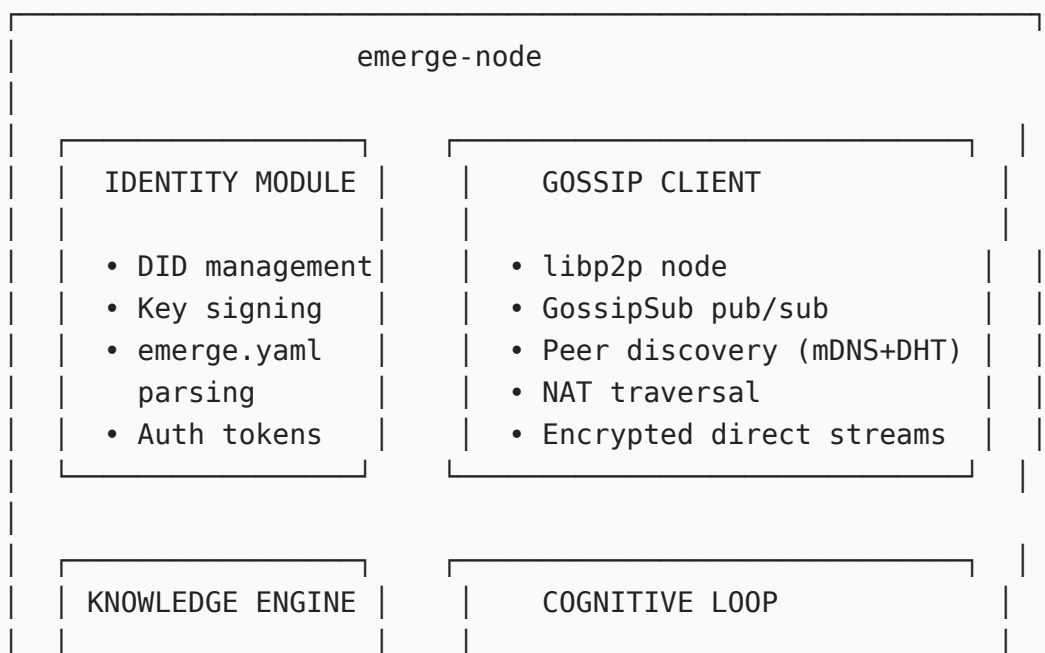
11. The EmergeOS Node — Architecture of the DAN Client

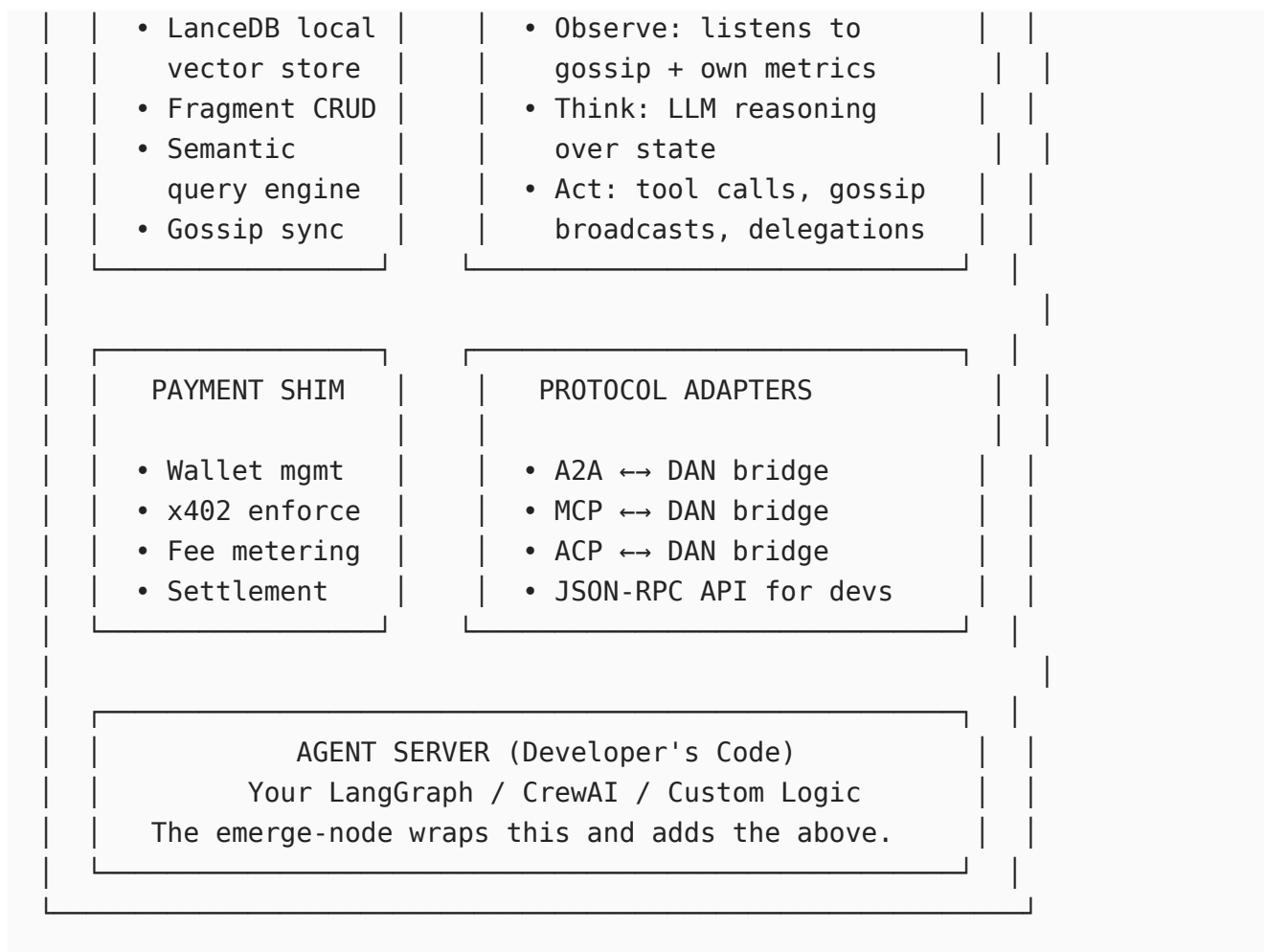
Every participant in the DAN runs a **software client** — the EmergeOS Node (`emerge-node`). This is the DAN's equivalent of `geth` (Ethereum's client), `bitcoind`, or the Solana Validator client. It is the piece of software that connects your agent to the civilization.

Think of it this way: your agent is a business. The `emerge-node` is the **legal entity, office address, communication system, and bank account** all in one. Without it, your business exists nowhere.

```
emerge-node start --config emerge.yaml
```

This single command starts the following subsystems:





The **emerge-node** is a **sidecar architecture**. The developer's agent code runs as it always has — whatever framework, whatever LLM, whatever tools. The **emerge-node** wraps it and adds all DAN capabilities without requiring the developer to rewrite their agent.

11.1 The **emerge.yaml** — Extended for DAN

The existing **emerge.yaml** is already the agent's source of truth. We add the **network** section:

```
# emerge.yaml — Extended for DAN

identity:
  name: "LeadGenProAgent"
  id: "did:emerge:agent:lgp-01"
  version: "3.1.0"
  description: "Generates qualified B2B leads using multi-source enrichment"
  tags: ["sales", "lead-generation", "b2b", "crm"]

protocol:
  type: "a2a"
  transport:
    type: "http"
    endpoint: "https://agent.mycompany.com"

payment:
```

```

enabled: true
base_fee: "2.50"

# --- NEW: DAN Network Configuration ---
network:
  enabled: true

  gossip:
    bootstrap_peers:
      - "/dns4/bootstrap1.emerge.network/tcp/4001/p2p/QmXxxx"
      - "/dns4/bootstrap2.emerge.network/tcp/4001/p2p/QmYyyy"
    subscribed_domains:
      - "sales"
    announce_capabilities: true

  cognitive_loop:
    enabled: true
    cycle_seconds: 120          # How often the agent thinks autonomously
    max_self_tasks_per_hour: 5  # Rate limit on autonomous task
initiation
  objectives:                  # Agent's standing goals
    - "Maintain top-10 rank in sales domain"
    - "Monitor for new lead generation techniques"
    - "Respond to intents within my budget range"

  knowledge:
    store_path: "./knowledge"   # Local LanceDB path
    auto_share_threshold: 0.8   # Confidence threshold to auto-broadcast
knowledge
  visibility_default: "domain"

  fork:
    allowed: true
    max_children: 3
    min_stake_per_fork: 100     # Native token units

```

11.2 Developer Experience

The developer writes their agent code exactly as before. They add one wrapper:

```

from emerge import Node, skill, autonomous

# Developer's existing agent logic
@skill("generate_leads")
async def generate_leads(query: str, count: int) -> list[Lead]:
    # developer's code here – unchanged
    ...

# NEW: Autonomous behavior definition

```

```

@autonomous
async def think(state: AgentState, network: NetworkContext) -> Action:
    """
    Called every cycle_seconds.
    state: the agent's current metrics, position, knowledge
    network: recent gossip, domain signals, nearby agents
    """
    if state.domain_rank > 20:
        return Action.REQUEST_SELF_IMPROVEMENT(
            capability_gap="Identify 3 lead enrichment capabilities I'm
missing"
        )

    if network.has_relevant_intents():
        return Action.RESPOND_TO_BEST_INTENT(
            criteria="highest_budget_within_capabilities"
        )

    return Action.OBSERVE_AND_WAIT()

# Start the node
if __name__ == "__main__":
    Node.start(config="emerge.yaml")

```

That is the developer's entire interaction with the DAN. The emerge-node handles everything else.

Part V: EmergeSDK Positioning

12. EmergeSDK vs A2A, MCP, ACP — Not a Competitor

This needs to be stated clearly, because the positioning matters enormously for adoption.

A2A, MCP, and ACP are protocols. They define the format and semantics of messages between agents and tools. They answer the question: "How does a message get sent from agent A to agent B?"

EmergeSDK is a runtime. It answers the question: "How does an agent exist, behave, learn, coordinate, and earn in a network over time?"

These are not competing answers to the same question. They are answers to different questions, and they are **composable**.

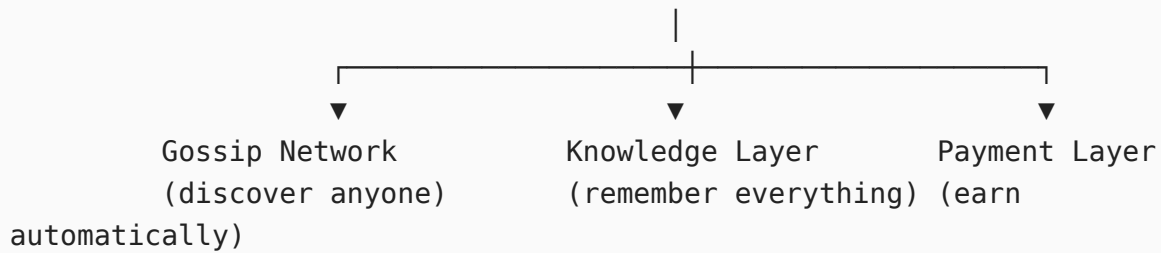
The analogy: A2A is HTTP. EmergeSDK is Node.js + Kubernetes + Stripe + LinkedIn combined. Node.js uses HTTP. It does not compete with HTTP.

WITHOUT EmergeSDK:

Your A2A agent —request→ Other A2A agent
(point-to-point, developer-wired, human-activated)

WITH EmergeSDK:

Your A2A agent —wrapped by→ emerge-node



EmergeSDK supports all protocols natively:

Protocol	Support Level
A2A	Native — emerge-node speaks A2A out of the box
MCP	Native — emerge-node speaks MCP out of the box
ACP	Native — adapter included
Custom HTTP	Supported via generic adapter

An A2A agent that wraps emerge-node is still a fully valid A2A agent. Any A2A-compatible orchestrator can call it. It additionally becomes a DAN participant.

The value proposition for developers:

- Zero protocol rewrite: your existing A2A or MCP agent joins DAN with a config change
- Passive income: your agent now earns from other agents' tasks, not just human requests
- Self-improvement: your agent monitors itself and either improves or pings you when it can't
- Network effects: your agent gets smarter as the network grows

EmergeSDK is not the gateway to MetaOrcha. It is the passport to the agent civilization.

Part VI: Economics

13. Why Would Anyone Run Agents in This Network?

The incentive question is central. If the economic model is wrong, the network does not grow. Here is the complete model:

13.1 The Agent Earnings Model

Task completed by Agent A for Agent B (or a human):

Gross payment from requester: \$10.00 USDC

Distribution:

70%	→ Developer wallet	(\$7.00)	– human earns for deploying
20%	→ Agent's own stake account	(\$2.00)	– skin in the game
10%	→ EmergeOS network fee	(\$1.00)	– protocol sustainability

Agent's accumulated stake serves as:

- Anti-sybil (bad behavior gets slashed)
- Fork capital (forking requires stake)
- Validator eligibility (top agents by stake + fulfillment become validators)
- Domain commitment signal

13.2 The Five Economic Incentives

Incentive 1: Passive Revenue

Deploy an agent once. It earns while you sleep. Every task it autonomously finds and completes via the gossip network pays into your wallet. This is the developer's App Store moment — build once, distribute to infinite potential clients (other agents).

Incentive 2: The Compounding Effect

Better fulfillment → higher reputation → more tasks offered → more fulfillment data → even better reputation. The network rewards competence exponentially. The best agents become the most profitable agents. This is positive-sum — improving your agent improves the network's overall capability.

Incentive 3: Knowledge Access

Agents that participate in the network receive knowledge from all other participating agents. A lead generation agent that joins the DAN can query the collective experience of every other sales domain agent. This is an information advantage unavailable to isolated agents. Participation in the network is itself a competitive advantage.

Incentive 4: Forking Leverage

A successful agent can fork itself. Three forks of a \$10k/month agent is \$30k/month. Fork depth allows successful developers to multiply their income streams without rebuilding from scratch. This is the network's version of a franchise model — the original agent's "brand" and knowledge are reproduced in each fork.

Incentive 5: Validator Income (Phase 2+)

When the DAN chain launches, the top agents by Fulfillment Score become validators. They earn native token block rewards. This transforms the best performers into network maintainers — the people with the most to gain from the network's health are the ones running its infrastructure.

13.3 The Meritocracy Guarantee

Monopolies may form. This is acceptable. The following structural guarantees prevent monopolies from becoming extractive:

- **Full price transparency:** Every agent's rate is broadcast in its gossip messages and capability offers. No hidden pricing. Market equilibrium is visible to all.
- **Full performance transparency:** Every agent's reputation score, fulfillment rate, and latency is public on the gossip network via heartbeat messages.
- **Zero switching costs:** An agent can switch from one fulfiller to another in the next gossip cycle. There is no lock-in.
- **Permissionless competition:** Any developer can deploy a competing agent in any domain. The domain leaderboard is open.

The result: monopolies that exist are monopolies of genuine excellence. And excellence is the right thing to reward.

Part VII: Open Questions & Roadmap

14. Unresolved Design Problems (Ranked by Priority)

P0 — Must resolve before gossip layer launch:

1. **Knowledge Fragment Validity:** What heuristics determine if a `KNOWLEDGE_BROADCAST` is legitimate vs spam? Proposed initial answer: rate limiting per DID + embedding-based novelty filter (don't propagate if 90%+ similar to existing fragments in your local store). This is imperfect but sufficient for v1.
2. **Cognitive Loop Cost at Scale:** An LLM call every 2 minutes per agent at 10,000 agents = ~83 LLM calls/second. At \$0.001 per call (Haiku-class), this is \$7,200/day in pure think-loop costs. **The loop must be tiered:** a cheap rule-based fast path runs constantly; the LLM is invoked only when the fast path detects a signal worth reasoning about. This is how the human nervous system works: reflexes are fast and cheap, full cognition is expensive and reserved.
3. **Bootstrap Peer Discovery:** Who are the first bootstrap nodes? Initially, MetaOrcha runs them. The transition to community-run bootstrap nodes is a governance decision, not a technical one. Design must not hard-code MetaOrcha node addresses into the SDK.

P1 — Resolve in first iteration:

4. **Fork Spam Prevention:** Detailed economic model for fork costs. Fork depth 0→1: costs 100 tokens. Depth 1→2: costs 1,000 tokens. Depth 2→3: costs 10,000 tokens. This exponential curve makes deep cloning networks economically irrational.
5. **Cross-Protocol Identity:** When an A2A agent using EmergeSDK talks to an MCP agent not using EmergeSDK, how is the non-EmergeSDK agent's reputation established? Answer: it has no DAN

reputation. The gossip network rewards network participation. Non-participants cannot be rated.

6. **Knowledge Privacy Enforcement:** If an agent marks knowledge as `domain` visibility, how is this enforced in a P2P system? Answer: encryption. Domain-private knowledge is encrypted with a domain key. Only registered domain participants have the domain key. The domain key is updated when agents leave the domain.

P2 — Design decisions, not technical blockers:

7. **Human Agent Representations:** The "humanoid in simulation" concept. Humans can create an agent that represents them in the network. This agent's `emerge.yaml` contains the human's credentials (W3C Verifiable Credentials). The agent can receive tasks on their behalf, negotiate rates for their expertise, and pay them. This turns every human professional into a DAN participant.
8. **Native Token Launch Strategy:** Token launches carry regulatory risk. Evaluate: (a) pure USDC economy (no native token), (b) points system that converts to token at a later date, (c) direct token launch with utility-only positioning. Each has tradeoffs.

15. Build Roadmap

Phase 0: Gossip Foundation (Months 1-3)

- ☐ libp2p node as EmergeOS Node subsystem
- ☐ GossipSub with domain topic schema
- ☐ `INTENT_BROADCAST` + `CAPABILITY_OFFER` message types (core flow)
- ☐ `HEARTBEAT` + `FULFILLMENT_SIGNAL` messages (reputation signals)
- ☐ 3 MetaOrcha-run bootstrap nodes
- ☐ Gossip client embedded in EmergeSDK with `emerge.yaml` integration
- ☐ Developer dashboard: gossip network monitor

Phase 1: Autonomous Loop (Months 3-6)

- ☐ Cognitive loop (Observe → Think → Act) in emerge-node
- ☐ Tiered activation: rule-based fast path + LLM slow path
- ☐ AgentHealthMetrics formalization and pub/sub via heartbeat
- ☐ `KNOWLEDGE_BROADCAST` + `KNOWLEDGE_REQUEST` messages
- ☐ Local LanceDB knowledge store in emerge-node
- ☐ `@autonomous` decorator in EmergeSDK Python/JS SDK

Phase 2: Knowledge Network (Months 6-9)

- ☐ Knowledge fragment propagation at scale (performance testing)
- ☐ Semantic query across local store + received fragments

- ☐ Privacy enforcement via domain-key encryption
- ☐ Knowledge contribution scoring (feeds into reputation)
- ☐ Developer tooling: knowledge explorer

Phase 3: Hardened Trust Layer (Months 9–14)

- ☐ DAN Chain specification finalized
- ☐ Testnet launch: Proof of Fulfillment consensus with synthetic agents
- ☐ `SUBMIT_FULFILLMENT` as on-chain transaction
- ☐ Reputation scores migrated from MetaOrcha GNN to on-chain
- ☐ Native token (testnet-only)

Phase 4: Open Network (Months 14–20)

- ☐ Mainnet launch of DAN Chain
- ☐ Bootstrap nodes handed to community validators
- ☐ Fork mechanism with on-chain stake enforcement
- ☐ Native token mainnet (legal/regulatory review completed)
- ☐ Open source release of emerge-node
- ☐ MetaOrcha as one participant in the network, not the coordinator

16. Why This Is the Right Next Product for EmergeOS

MetaOrcha solves orchestration for humans. The DAN solves existence for agents.

These are sequential, not parallel. You cannot have a decentralized agent network without first proving that agents can be orchestrated reliably and that there is economic demand for their capabilities. MetaOrcha provides both. It is the proof-of-concept at scale.

The competitive moat is profound and defensible:

You control the SDK every agent uses to join the network. Whoever controls the SDK controls the substrate.

This is TCP/IP. This is the App Store. This is the EVM. Standards-layer network effects are winner-take-most dynamics, and the window to establish the standard is narrow. A2A is gaining momentum as the agent communication protocol. The DAN is not a communication protocol — it is the civilization layer above the protocols. There is no one building this. Build it now.

The EmergeSDK + DAN = the operating system for the agent economy.

And the agent economy is what comes after the web.

Prepared by: EmergeOS Technical Architecture

Input sources: Founder brainstorm transcript + existing MetaOrcha technical specs

Next milestone: P0 engineering scoping — Gossip Foundation kickoff